

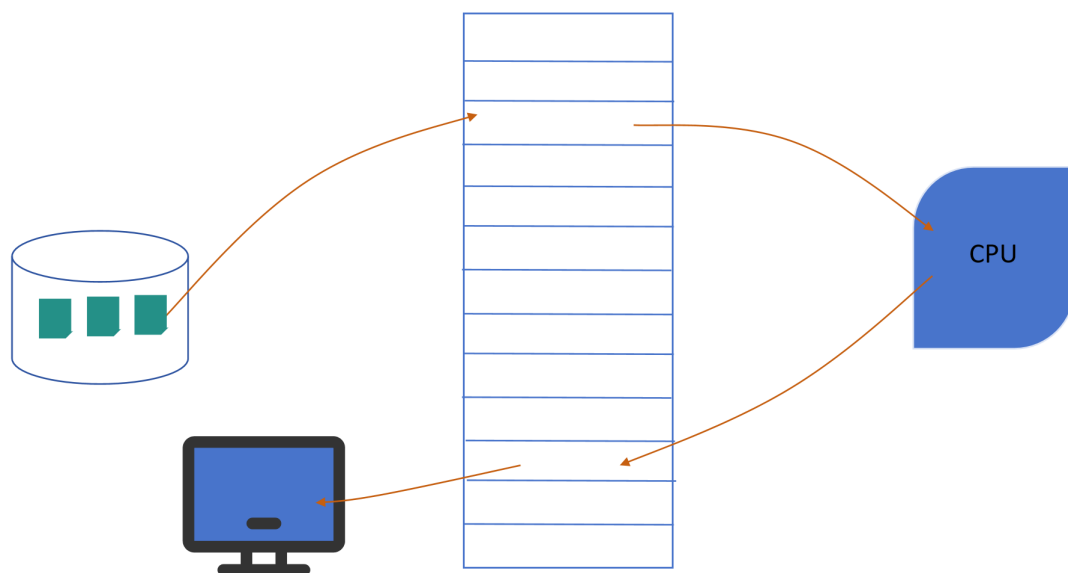
第1章 计算机基础知识

1.1 计算机运行基本原理

CPU是中央处理单元的意思，它在计算机里的作用就像人的大脑一样。人的一切行为（条件反射

的除外）都是由大脑支配的，计算机里的绝大部分指令都是通过CPU来执行的。所以，编程就是

把指令写好，放在内存里，让CPU去执行。



➡ 此处解释一下编译，和脚本代码。

➡ 如何编译

1.2 计算机的文件

文件类型分为2种——可执行文件（包含指令文件）和数据文件。在Windows中，可执行文件的后缀是exe（也有.bat，但bat实际是脚本）。

➡ 在Windows下演示可执行文件和数据文件的区别。还要解释一下C语言的主函数的参数。

```

1  #include <stdio.h>
2  int main(int arg ,char** args){
3
4      printf("第一个参数: %d\n",arg);
5      if(args == NULL){
6          return 0;
7      }
8      int i = 0;
9      for(;i < arg;i++){
10         printf("第%d个参数: %s\n",i,args[i]);
11     }
12     return 0;
13 }

```

☞ 双击的时候，可执行程序是怎么直接打开文件的。

1.3 可视化的编程

用dev c++演示可视化的编程。

1.4 讲解动态链接库

简单介绍一下链接库的知识

1.5 内存分配机制

```

1  // example8_4.c
2  #include <stdio.h>
3  int global = 0;
4  void position(int pos_a){
5      int pos_b = 0;
6      static int pos_s = 0;
7      int *pos_m = (int*)malloc(sizeof(int));
8      printf("pos_a:%x\n",&pos_a);           //函数参数地址（局部）
9      printf("pos_b:%x\n",&pos_b);           //函数局部变量地址
10     printf("pos_s:%x\n",&pos_s);           //函数静态变量地址
11     printf("pos_m:%x\n",pos_m);             //函数中动态分配的内存地址
12 }
13
14 int main()
15 {
16     int main_a = 0;
17     static int main_s = 0;
18     int *main_m = (int*)malloc(sizeof(int));
19     printf("main_a:%x\n",&main_a);           //main函数局部变量地址
20     printf("global:%x\n",&global);           //全局变量地址
21     printf("main_s:%x\n",&main_s);           //main函数静态变量地址

```

```

22     printf("main_m:%x\n",main_m);           //main函数中动态分配的内存
地址
23     printf("function main:%x\n",main);       //main函数地址
24     printf("function position:%x\n",position); //函数地址
25     position(0);                             //执行 position函数
26     return 0;
27 }

```

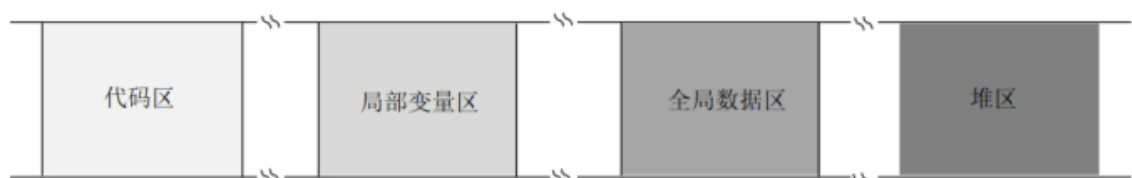
(1) 代码区。example8_1.c有两个函数组成，每个函数都会最终被编译成机器指令。运行时，这些指令是要加载到内存中的，而函数名就是这些指令在内存中的首地址，所以main、position都是指针，main指向的是main函数的指令块首地址，position指向的是position指令块的首地址。

(2) 局部变量区。无论在main函数中，还是在position函数中，局部变量似乎是存在一起的——包括函数参数。不同的是，随着position函数运行结束，内部的局部变量生命也就结束了，其占用的内存也会被释放，而main函数中的局部变量会在程序结束时释放。

(3) 全局数据区域。无论在main函数中，还是在position函数中，静态变量是存储在一起的，它们又和全局变量处在同一个区域。此区域的数据为全局共享，其占有的内存不会随着函数运行结束而释放，只有待程序运行结束才会释放。

(4) 堆区域。动态分配的内存位于堆区域，这段内存也是全程序共享的，且在程序结束时才会被释放（或者由程序开发人员主动释放）。例如，虽然pos_m指针是在position函数中动态分配的，但position运行结束后，并不会主动释放。

根据上述分析，一段程序运行其内存地址大致可分为4段（图8-8）。



➡ 强调，返回的指针千万不能定义在函数中